Dice Rolling Simulator

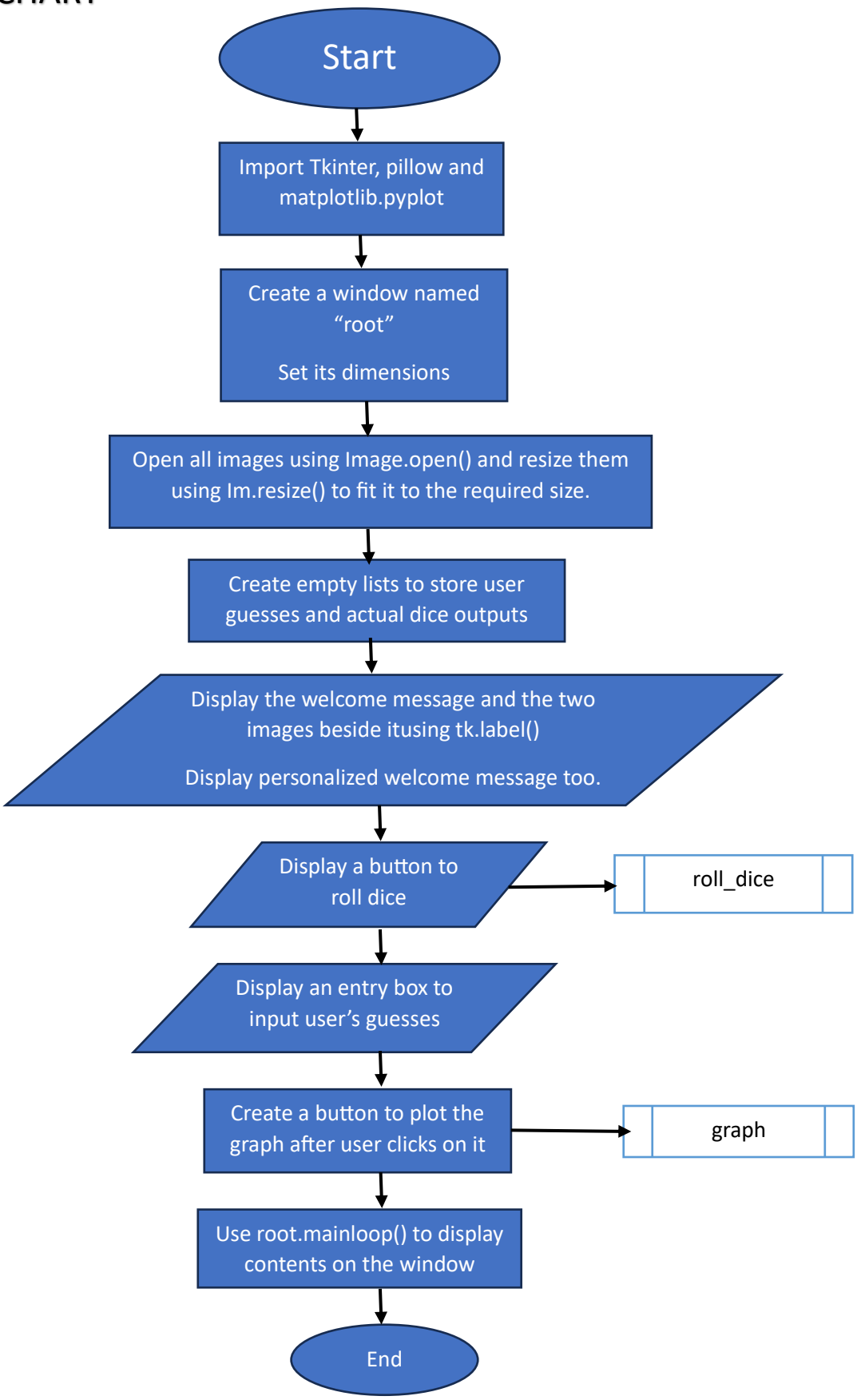
- The project is a Dice Rolling Simulator implemented using Python's Tkinter library for the graphical user interface (GUI).

- It allows users to input their guess for the outcome of rolling a dice, rolls the dice randomly, compares the user's guess with the actual outcome, displays the result, and provides an option to generate a graph comparing user guesses with actual dice values.

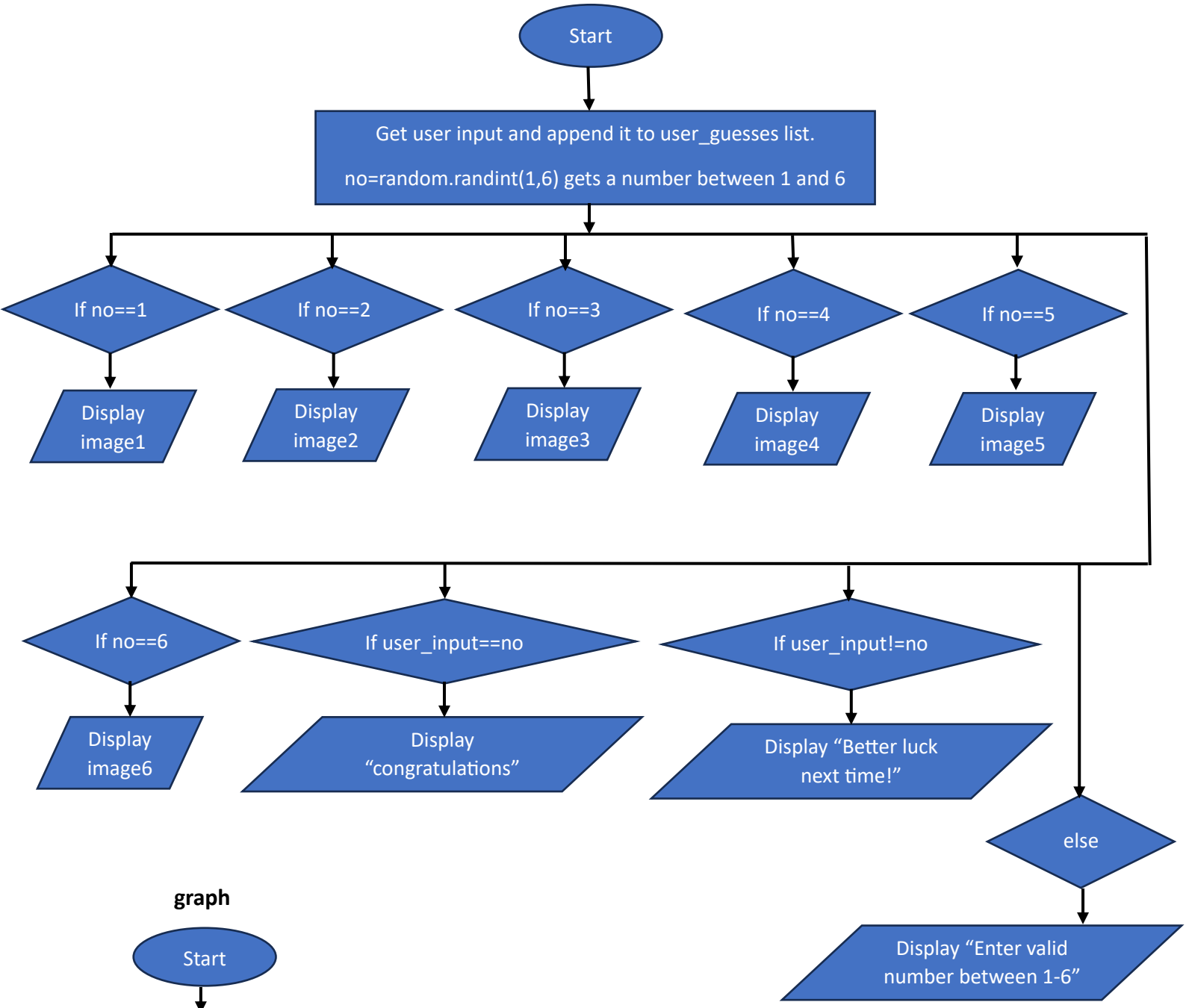
- The code includes image handling for displaying dice faces, error handling for invalid user inputs, and graph plotting using Matplotlib.

- Overall, it provides an interactive way for users to simulate dice rolling and analyse their guesses.

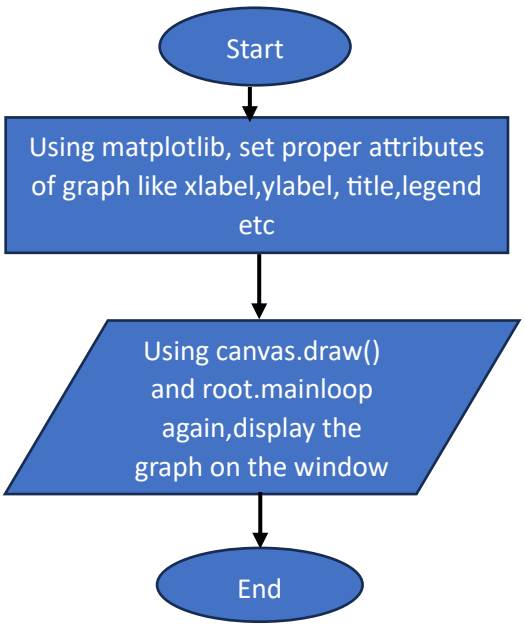
FLOWCHART


Start
Import Tkinter, pillow and matplotlib.pyplot
Create a window named "root"
Set its dimensions
Open all images using Image.open() and resize them using Im.resize() to fit it to the required size.
Create empty lists to store user guesses and actual dice outputs
Display the welcome message and the two images beside itusing tk.label()
Display personalized welcome message too.
Display a button to roll dice
roll_dice
Display an entry box to input user's guesses
Create a button to plot the graph after user clicks on it
graph
Use root.mainloop() to display contents on the window
End

roll_dice



graph



Code

```
import tkinter as tk
from PIL import Image, ImageTk
import random
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import (FigureCanvasTkAgg)


root = tk.Tk()
root.geometry("1000x800")
root.configure(bg="black")
root.title("Dice Rolling Simulator ")

# Read the images
image1 = Image.open("dice1_.jpeg")
image2 = Image.open("dice2_.jpeg")
image3 = Image.open("dice3_.jpeg")
image4 = Image.open("dice4_.jpeg")
image5 = Image.open("dice5_.jpeg")
image6 = Image.open("dice6_.jpeg")
image7 = Image.open("dice.jpeg")

# Resize the images
resize_image1 = image1.resize((170, 170))
resize_image2 = image2.resize((170, 170))
resize_image3 = image3.resize((170, 170))
resize_image4 = image4.resize((170, 170))
resize_image5 = image5.resize((170, 170))
resize_image6 = image6.resize((170, 170))
resize_image7 = image7.resize((50,50))

user_guesses = [] #Creating empty list to store user guesses
correct_guesses = [] #Creating empty list to store actual dice values

Font_tuple = ("Comic Sans MS", 25, "bold")
Font_2_tuple= ("Comic Sans MS", 17)

#Defining a function to get user input, append each input to the list, picking a
#random number and generating the right image accordingly
def roll_dice():
    user_input =int(r.get())
    user_guesses.append(user_input)
    no = random.randint(1, 6)
    correct_guesses.append(no)

    if no == 1:
        image_dice=ImageTk.PhotoImage(resize_image1)
            label1=tk.Label(root,image=image_dice)
```

```
        label1.place(x=430,y=380)

    if no == 2:
        image_dice=ImageTk.PhotoImage(resize_image2)
        label2=tk.Label(root,image=image_dice)
        label2.place(x=430,y=380)
    if no == 3:
        image_dice=ImageTk.PhotoImage(resize_image3)
        label3=tk.Label(root,image=image_dice)
        label3.place(x=430,y=380)
    if no == 4:
        image_dice=ImageTk.PhotoImage(resize_image4)
        label4=tk.Label(root,image=image_dice)
        label4.place(x=430,y=380)
    if no == 5:
        image_dice=ImageTk.PhotoImage(resize_image5)
        label5=tk.Label(root,image=image_dice)
        label5.place(x=430,y=380)
    if no == 6:
        image_dice=ImageTk.PhotoImage(resize_image6)
        label6=tk.Label(root,image=image_dice)
        label6.place(x=430,y=380)

    if user_input < 1 or user_input > 6:
        result_label = tk.Label(root, text="Invalid Input!! Please Try another input from 1 to 6 :)
:) :)", font="20", bg="red")
        result_label.place(x=650, y=450)

    elif user_input==no :
        result_label = tk.Label(root, text="Congratulations!! You guessed the number correctly
!",font="15",bg="gold")
        result_label.place(x=650, y=450)

    else :
        result_label = tk.Label(root, text="Ohh No!!Better luck next
time.............................!",font="15",bg="gold")
        result_label.place(x=650, y=450)


    root.mainloop()

 # Plotting the graph
def graph():
    fig, plot1=plt.subplots()
    canvas1=FigureCanvasTkAgg(fig, master=root)
    canvas1.get_tk_widget().pack(padx=80, pady=70)
    plot1.plot(user_guesses, label='User Guesses', color='blue', marker='o')
    plot1.plot(correct_guesses, label='Actual Dice Value', color='red', marker='x')
```

```
    plt.grid(color = 'darkslategrey', linestyle = '--', linewidth = 0.5)

    plt.xlabel(' Dice rolled Number')
    plt.ylabel('Dice Value')
    plt.title('User Guesses vs Actual Dice Value')
    plt.legend()
    plt.grid(True)

    canvas1.draw()
    root.mainloop()
def info():
# Get user's name from the entry widget
    user_name = entry_name.get()
# Create a label to display the welcome message
    welcome_message = "Hello {}, Here is our Dice Roll Simulator. You can enter your guess
in the following label and roll the dice.\n If you guess it right, a congratulations window will
appear. Otherwise, you can give it another try. \nAt the end, don't forget to generate your
graph. It will help you to analyze your results.".format(user_name)
    welcome_label = tk.Label(root, text=welcome_message,wraplength=1300)
    welcome_label.place(x=90, y=154)
    welcome_label.configure(font=Font_2_tuple,bg="black",fg="cyan")

imaget=ImageTk.PhotoImage(resize_image7)
label7=tk.Label(root,image=imaget)
label7.place(x=290,y=10)
#Setting the style of the heading with proper colour, size and alignment attributes
label_welc=tk.Label(root,text="Welcome to Dice Roll
Simulator!",bg="black",font=Font_tuple)
label_welc.place(x=300,y=0)
label_welc.config(fg="Yellow")
label_welc.pack(anchor=tk.CENTER)
imagee=ImageTk.PhotoImage(resize_image7)
label8=tk.Label(root,image=imagee)
label8.place(x=947,y=10)

#Asking user to input their name to provide a personalised touch
lable_name=tk.Label(root,text="Please enter your name: ",font=Font_2_tuple)
lable_name.place(x=440,y=66)
lable_name.config(fg="lime",bg="black")
entry_name = tk.Entry(root)
entry_name.place(x=710, y=76)

button_next = tk.Button(root, text="NEXT",
command=info,activebackground="green",activeforeground="yellow",padx
="2",pady="2",font='5')
button_next.place(x=630, y=110)
#Field to input user choice
```

```
input_label = tk.Label(root, text="Enter Your Choice (Number between
1 to 6)   >>>",font="15")
input_label.place(x=335, y=270)
input_label.config(fg="lime",bg="black")
r = tk.Entry(root)
r.place(x=735, y=275)

#Button which calls the roll_dice function
button = tk.Button(root, text="ROLL THE DICE",
command=roll_dice,activebackground="green",activeforeground="yellow",padx="3",pady="
3",font='7')
button.place(x=590, y=310)

#Buttons to generate the graph
graph_label=tk.Label(root,text="Click here to Generate the
Graph",font=Font_2_tuple,fg="lime",bg="black")
graph_label.place(x=300,y=580)
button = tk.Button(root, text="Plot the graph",
command=graph,activebackground="green",activeforeground="yellow",padx="3",pady="3",
font='7',relief="sunken")
button.place(x=700, y=577)

root.mainloop() #Displays all attributes on the tkinter window.
```

OUTPUT

1) The initial window of the program is displayed when the user runs the program.

2) After entering their name, the user interface prompts for their guess.

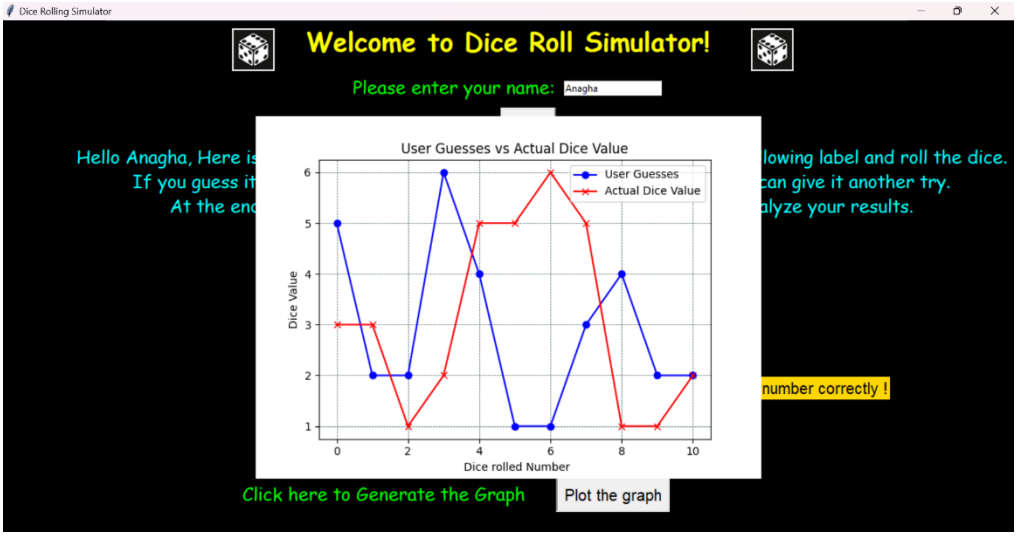
1

3) When the user correctly guesses the value, a congratulatory message is shown.

4) If the user makes an incorrect guess, a message indicating so is displayed.

5) When the input provided by the user is invalid, an error message appears.

6) Upon generating the graph, a graphical representation of user guesses versus actual dice value is displayed.



REFERENCES:

1) Please refer to these images for easy execution of 'Dice Roller' and make sure to download these images in your python mini project folder before executing the code.
   https://docs.google.com/document/d/1SkSDveC01skvbyabVwmimc5aWid4g7H_J0QcSax8nNE/edit?usp=sharing

1) Reference websites:
   2.1 https://www.w3schools.com/python/default.asp
   2.2 https://www.geeksforgeeks.org/python-gui-tkinter/

2) Reference Books:
   - Fredrik Johansson et al., "Mpmath: A Python library for Arbitrary-Precision Floating Point Arithmetic"